

GSOI 2026 暑期常规赛

第九试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 五轮启幽扉 (lock)

T2 双径会深庭 (cross)

T3 星脉聚流光 (energy)

T4 重彩书遗年 (paint)

吐槽时间

Contents

T1 五轮启幽扉 (lock)

T2 双径会深庭 (cross)

T3 星脉聚流光 (energy)

T4 重彩书遗年 (paint)

吐槽时间

数据点 1 ~ 5 25 Pts

- 特殊性质 A 保证所有状态两两不同，并且只有同一个转轮上的数字会发生变化。因为 $n \geq 3$ ，若原始密钥还在其他转轮上与记录不同，那么每次试拨都必须同时改变这个转轮和变化的转轮，两个转轮上的差应当始终相同，这与记录中只有一位发生变化矛盾。因此原始密钥的其余四位已经被记录唯一确定。

数据点 1 ~ 5 25 Pts

- ▶ 特殊性质 A 保证所有状态两两不同，并且只有同一个转轮上的数字会发生变化。因为 $n \geq 3$ ，若原始密钥还在其他转轮上与记录不同，那么每次试拨都必须同时改变这个转轮和变化的转轮，两个转轮上的差应当始终相同，这与记录中只有一位发生变化矛盾。因此原始密钥的其余四位已经被记录唯一确定。
- ▶ 变化转轮的原始数字可以是 $0 \sim 9$ 中任意一个没有在记录中出现的数字，出现过的数字会使某条记录与原始密钥相同。答案就是 $10 - n$ 。检查变化位置需要 $\mathcal{O}(n)$ 的时间，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 6 ~ 10 50 Pts

- ▶ 特殊性质 B 中，全部记录只在一个固定的相邻转轮对上变化，并且两位变化量始终相同。于是这些状态位于模 10 意义下的一条长度为 10 的轨道上。与上一档相同，原始密钥必须位于这条轨道上，并且不能等于已经给出的状态。

数据点 6 ~ 10 50 Pts

- ▶ 特殊性质 B 中，全部记录只在一个固定的相邻转轮对上变化，并且两位变化量始终相同。于是这些状态位于模 10 意义下的一条长度为 10 的轨道上。与上一档相同，原始密钥必须位于这条轨道上，并且不能等于已经给出的状态。
- ▶ 因为记录两两不同，所以答案仍然为 $10 - n$ 。这一档保留了上一档“只比较模差”的判断，只把一个变化位置推广成相邻两个变化量相同的位置，时间复杂度仍为 $\mathcal{O}(n)$ 。

数据点 11 ~ 15 75 Pts

- ▶ 这一档不再给出满足的是特殊性质 A 还是 B。考虑比较第一条记录与其余记录，若所有非零差都集中在同一位，就使用第一档的单轮轨道。否则寻找唯一的相邻轮对，并检查两位模差是否始终相同，再使用第二档的双轮轨道。

数据点 11 ~ 15 75 Pts

- ▶ 这一档不再给出满足的是特殊性质 A 还是 B。考虑比较第一条记录与其余记录，若所有非零差都集中在同一位，就使用第一档的单轮轨道。否则寻找唯一的相邻轮对，并检查两位模差是否始终相同，再使用第二档的双轮轨道。
- ▶ 两种性质下的答案都是 $10 - n$ 。这一档的瓶颈是一般数据可能同时包含由不同单轮或不同相邻轮对得到的记录，无法再把候选压缩到同一条轨道，但模差的合法性判断仍然可以原样保留。

正解 100 Pts

- ▶ 五个转轮均只有 10 种取值，所以完整密钥一共只有 10^5 种。直接枚举五位数字。对于一条记录，计算候选密钥到记录的五个模 10 差。差全部为零时没有进行试拨，恰有一个非零差时对应单轮试拨，恰有两个非零差时要求它们位置相邻且数值相等，其余情况均不合法。

正解 100 Pts

- ▶ 五个转轮均只有 10 种取值，所以完整密钥一共只有 10^5 种。直接枚举五位数字。对于一条记录，计算候选密钥到记录的五个模 10 差。差全部为零时没有进行试拨，恰有一个非零差时对应单轮试拨，恰有两个非零差时要求它们位置相邻且数值相等，其余情况均不合法。
- ▶ 每个候选都通过全部 n 条记录的检查时才计入答案，因此不会遗漏或重复任何原始密钥。特别地， $n = 1$ 时一条记录反向进行一次试拨恰有 $5 \times 9 + 4 \times 9 = 81$ 个候选。总时间复杂度为 $\mathcal{O}(10^5 n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

```
1 bool Try(int a,int b,int c,int d,int e){
2     for(int i=1;i<=n;i++){
3         int Da=0,Db=0,Dc=0,Dd=0,De=0,Times=0;
4         Da=(a-Condition[i][1]+10)%10;
5         if(Da) Times++;
6         Db=(b-Condition[i][2]+10)%10;
7         if(Db) Times++;
8         Dc=(c-Condition[i][3]+10)%10;
9         if(Dc) Times++;
10        Dd=(d-Condition[i][4]+10)%10;
11        if(Dd) Times++;
12        De=(e-Condition[i][5]+10)%10;
13        if(De) Times++;
```

正解 100 Pts

```
1      if(!Times) return false;
2      if(Times==1) continue;
3      if(Times==2){
4          if((Da==Db&&Da!=0) || (Db==Dc&&Db!=0) || (Dc==Dd&&Dc!=0) || (Dd==
De&&Dd!=0)) continue;
5          else return false;
6      }
7      if(Times>2) return false;
8  }
9  return true;
10 }
```

正解 100 Pts

```

1 void Solve(){
2     scanf("%d",&n);
3     Ans=0;
4     for(int i=1;i<=n;i++){
5         for(int j=1;j<=5;j++){
6             scanf("%d",&Condition[i][j]);
7         }
8     }
9     if(n==1) printf("81\n");
10    else{

```

正解 100 Pts

```

1      for(int i=0;i<=9;i++){
2          for(int j=0;j<=9;j++){
3              for(int k=0;k<=9;k++){
4                  for(int r=0;r<=9;r++){
5                      for(int o=0;o<=9;o++){
6                          if(Try(i,j,k,r,o)){
7                              Ans++;
8                          }
9                      }
10                 }
11             }
12         }
13     }
14     printf("%d\n",Ans);
15 }
16

```

Contents

T1 五轮启幽扉 (lock)

T2 双径会深庭 (cross)

T3 星脉聚流光 (energy)

T4 重彩书遗年 (paint)

吐槽时间

数据点 1 ~ 5 25 Pts

- ▶ 对一次询问分别从 s_1 与 s_2 出发搜索，恢复两条简单路径，并标记路径上的全部结点。扫描全部结点，只要存在一个结点同时属于两条路径，答案就是 Yes。这一做法完全按照相交的定义判断，所以充要性是直接的。

数据点 1 ~ 5 25 Pts

- ▶ 对一次询问分别从 s_1 与 s_2 出发搜索，恢复两条简单路径，并标记路径上的全部结点。扫描全部结点，只要存在一个结点同时属于两条路径，答案就是 Yes。这一做法完全按照相交的定义判断，所以充要性是直接的。
- ▶ 每次询问需要两次搜索和一次扫描，时间复杂度为 $\mathcal{O}(nq)$ ，空间复杂度为 $\mathcal{O}(n)$ 。当 n, q 同时增大后，重复恢复路径成为瓶颈。

数据点 6 ~ 10 50 Pts

- 特殊性质保证整棵树是一条链。找到度数为 1 的端点后沿链确定每个结点的位置，路径 (s, t) 就变成位置轴上的闭区间 $[\min(\text{pos}_s, \text{pos}_t), \max(\text{pos}_s, \text{pos}_t)]$ ，两条路径相交当且仅当两个闭区间相交。

数据点 6 ~ 10 50 Pts

- ▶ 特殊性质保证整棵树是一条链。找到度数为 1 的端点后沿链确定每个结点的位置，路径 (s, t) 就变成位置轴上的闭区间 $[\min(\text{pos}_s, \text{pos}_t), \max(\text{pos}_s, \text{pos}_t)]$ ，两条路径相交当且仅当两个闭区间相交。
- ▶ 预处理需要 $\mathcal{O}(n)$ 的时间，每次询问为 $\mathcal{O}(1)$ 。这一档说明不必恢复路径本身，只要能够用少量端点信息判定某个结点是否位于路径上即可。

数据点 11 ~ 15 75 Pts

- ▶ 这一档只有 $q \leq 300$ ，仍然沿用第一档的路径标记，时间复杂度为 $\mathcal{O}(nq)$ 。与第一档相比，树的规模已经达到完整范围，但询问数量限制了重复搜索的总次数。

数据点 11 ~ 15 75 Pts

- ▶ 这一档只有 $q \leq 300$ ，仍然沿用第一档的路径标记，时间复杂度为 $\mathcal{O}(nq)$ 。与第一档相比，树的规模已经达到完整范围，但询问数量限制了重复搜索的总次数。
- ▶ 正解需要去掉对 q 的限制。根据上一档部分的启发，我们只需要支持两件事：求两个端点的路径中深度最小的结点，以及用距离判断一个结点是否位于路径上。

正解 100 Pts

- 任选结点 1 为根，倍增预处理 LCA，并用 $\text{dist}(u, v) = \text{dep}_u + \text{dep}_v - 2 \text{dep}_{\text{LCA}(u, v)}$ 计算距离。结点 x 位于路径 (u, v) 上，当且仅当 $\text{dist}(u, x) + \text{dist}(x, v) = \text{dist}(u, v)$ 。

正解 100 Pts

- ▶ 任选结点 1 为根，倍增预处理 LCA，并用 $\text{dist}(u, v) = \text{dep}_u + \text{dep}_v - 2 \text{dep}_{\text{LCA}(u, v)}$ 计算距离。结点 x 位于路径 (u, v) 上，当且仅当 $\text{dist}(u, x) + \text{dist}(x, v) = \text{dist}(u, v)$ 。
- ▶ 令 $p_1 = \text{LCA}(s_1, t_1)$, $p_2 = \text{LCA}(s_2, t_2)$ 。两条路径相交当且仅当 p_1 位于第二条路径上，或者 p_2 位于第一条路径上。这样每次询问只需要常数次 LCA。

正解 100 Pts

- 充分性是直接的。考虑必要性，假设两条路径相交，并令 z 为交集中深度最小的结点。因为 z 分别位于两条路径上，所以 p_1, p_2 都是 z 的祖先，两者一定具有祖先关系。不妨设 p_1 更深，那么 p_1 位于从 p_2 到 z 的祖先链上，而这条链完全属于第二条路径，所以 p_1 位于第二条路径上。

正解 100 Pts

- ▶ 充分性是直接的。考虑必要性，假设两条路径相交，并令 z 为交集中深度最小的结点。因为 z 分别位于两条路径上，所以 p_1, p_2 都是 z 的祖先，两者一定具有祖先关系。不妨设 p_1 更深，那么 p_1 位于从 p_2 到 z 的祖先链上，而这条链完全属于第二条路径，所以 p_1 位于第二条路径上。
- ▶ 因此上述判定是充要的。预处理时间复杂度为 $\mathcal{O}(n \log n)$ ，每次询问为 $\mathcal{O}(\log n)$ ，总时间复杂度为 $\mathcal{O}((n + q) \log n)$ ，空间复杂度为 $\mathcal{O}(n \log n)$ 。

正解 100 Pts

```
1 int LCA(int a,int b){
2     ;;;;if(Depth[a]<Depth[b]) swap(a,b);
3     ;;;;for(int i=19;i>=0;i--){
4         ;;;;;;;;;;if(Depth[Fa[a][i]]>=Depth[b]) a=Fa[a][i];
5     ;;;;}
6     ;;;;if(a==b) return a;
7     ;;;;for(int i=19;i>=0;i--){
8         ;;;;;;;;;;if(Fa[a][i]!=Fa[b][i]){
9             ;;;;;;;;;;a=Fa[a][i];
10            ;;;;;;;;;;b=Fa[b][i];
11        ;;;;;;;;;;}
12    ;;;;}
13    ;;;;return Fa[a][0];
14 }
15 int Get(int a,int b){
16     ;;;;int lca=LCA(a,b);
17     ;;;;return Depth[a]+Depth[b]-2*Depth[lca];
```

正解 100 Pts

```
1 bool Query(int a,int b,int c,int d){
2     ;;;;int Lca1=LCA(a,b);
3     ;;;;int Lca2=LCA(c,d);
4     ;;;;int Dis1=Get(a,b);
5     ;;;;int Dis2=Get(c,d);
6     ;;;;int Judge1=Get(Lca1,c)+Get(Lca1,d);
7     ;;;;int Judge2=Get(Lca2,a)+Get(Lca2,b);
8     ;;;;if(Judge1==Dis2||Judge2==Dis1) return true;
9     // ;;;;if(LCA(Lca1,Lca2)==Lca2&&(LCA(Lca1,d)==Lca1||LCA(Lca1,c)==Lca1))
10         return true;
11 // ;;;;if(LCA(Lca1,Lca2)==Lca1&&(LCA(Lca2,a)==Lca2||LCA(Lca2,b)==Lca2))
12         return true;
13 ;;;;return false;
14 }
```

正解 100 Pts

```
1 void Solve(){
2     ;;;int n,q;
3     ;;;scanf("%d%d",&n,&q);
4     ;;;Total=0;
5     ;;;memset(Head,0,sizeof(int)*(n+1));
6     ;;;memset(Depth,0,sizeof(int)*(n+1));
7     ;;;memset(Fa,0,sizeof(int)*(n+1)*21);
8     ;;;for(int i=1;i<n;i++){
9         ;;;scanf("%d%d",&In1,&In2);
10        ;;;Edge_Add(In1,In2);
11        ;;;Edge_Add(In2,In1);
12    ;;;}
```

正解 100 Pts

```
1  ;;;;DFS(1,1);
2  ;;;;for(int i=1;i<=19;i++){
3  ;;;;;;;;;;for(int j=1;j<=n;j++){
4  ;;;;;;;;;;;;;Fa[j][i]=Fa[Fa[j][i-1]][i-1];
5  ;;;;;;;;;;}
6  ;;;;}
7  ;;;;while(q-->0){
8  ;;;;;;;;;;scanf("%d%d%d%d",&In1,&In2,&In3,&In4);
9  ;;;;;;;;;;printf("%s\n",Query(In1,In2,In3,In4)?"Yes":"No");
10 ;;;;}
```

Contents

T1 五轮启幽扉 (lock)

T2 双径会深庭 (cross)

T3 星脉聚流光 (energy)

T4 重彩书遗年 (paint)

吐槽时间

数据点 1 ~ 6 24 Pts

- 从 $(0, 0)$ 到 (x, y) 的线段内部共有 $\gcd(x, y) - 1$ 个整点，所以这个结点的传输损失为 $2\gcd(x, y) - 1$ 。若 $\min(n, m) = 1$ ，每个点的损失都是 1，答案为 $\max(n, m)$ 。若 $\min(n, m) = 2$ ，记 $s = \max(n, m)$ ，第一行贡献 s ，第二行在坐标为偶数时额外贡献 2，答案为 $2s + 2\lfloor s/2 \rfloor$ 。

数据点 1 ~ 6 24 Pts

- ▶ 从 $(0,0)$ 到 (x,y) 的线段内部共有 $\gcd(x,y) - 1$ 个整点，所以这个结点的传输损失为 $2\gcd(x,y) - 1$ 。若 $\min(n,m) = 1$ ，每个点的损失都是 1，答案为 $\max(n,m)$ 。若 $\min(n,m) = 2$ ，记 $s = \max(n,m)$ ，第一行贡献 s ，第二行在坐标为偶数时额外贡献 2，答案为 $2s + 2\lfloor s/2 \rfloor$ 。
- ▶ 这一档只需常数时间。它已经把几何问题转化成最大公约数之和，后续只需要更快地统计不同最大公约数出现的次数。

数据点 7 ~ 12 48 Pts

- ▶ 直接枚举全部 $n \times m$ 个结点，使用辗转相除法计算 $\gcd(x, y)$ ，再累加 $2 \gcd(x, y) - 1$ 。每个结点被恰好计算一次，所以算法显然正确。

数据点 7 ~ 12 48 Pts

- ▶ 直接枚举全部 $n \times m$ 个结点，使用辗转相除法计算 $\gcd(x, y)$ ，再累加 $2 \gcd(x, y) - 1$ 。每个结点被恰好计算一次，所以算法显然正确。
- ▶ 时间复杂度为 $\mathcal{O}(nm \log \min(n, m))$ ，空间复杂度为 $\mathcal{O}(1)$ 。这一做法的瓶颈是逐点处理，而相同最大公约数的点对可以放在一起统计。

数据点 13 ~ 19 76 Pts

- 这一档满足 $n = m$ 。设 $\Phi(s) = \sum_{i=1}^s \varphi(i)$ ，那么 $[1, s]^2$ 中有 $2\Phi(s) - 1$ 个有序互质点对。枚举最大公约数 d ，将两个坐标同时除以 d ，最大公约数恰为 d 的点对数量就是 $2\Phi(\lfloor n/d \rfloor) - 1$ 。

数据点 13 ~ 19 76 Pts

- ▶ 这一档满足 $n = m$ 。设 $\Phi(s) = \sum_{i=1}^s \varphi(i)$ ，那么 $[1, s]^2$ 中有 $2\Phi(s) - 1$ 个有序互质点对。枚举最大公约数 d ，将两个坐标同时除以 d ，最大公约数恰为 d 的点对数量就是 $2\Phi(\lfloor n/d \rfloor) - 1$ 。
- ▶ 线性筛预处理 Euler 函数及其前缀和，再枚举 d 计算贡献。必要性来自把坐标除以它们的最大公约数，充分性来自把任意互质点对同时乘以 d 。时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。正解继续使用“按照最大公约数分类”的思路处理矩形。

正解 1 100 Pts

- 令 $r = \min(n, m)$, 定义 f_d 为满足 $1 \leq x \leq n$ 、 $1 \leq y \leq m$ 且 $\gcd(x, y) = d$ 的有序点对数量。坐标都能被 d 整除的点对共有 $\lfloor n/d \rfloor \lfloor m/d \rfloor$ 个, 其中还包含最大公约数为 $2d, 3d, \dots$ 的点对。

正解 1 100 Pts

- ▶ 令 $r = \min(n, m)$, 定义 f_d 为满足 $1 \leq x \leq n$ 、 $1 \leq y \leq m$ 且 $\gcd(x, y) = d$ 的有序点对数量。坐标都能被 d 整除的点对共有 $\lfloor n/d \rfloor \lfloor m/d \rfloor$ 个, 其中还包含最大公约数为 $2d, 3d, \dots$ 的点对。
- ▶ 因此从大到小枚举 d , 完整递推式为

$$f_d = \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor - \sum_{\substack{k \geq 2 \\ kd \leq r}} f_{kd}.$$

枚举顺序保证右侧状态都已经求出, 不存在的状态视为 0。

正解 1 100 Pts

- 每一对坐标恰好归入唯一的 f_d ，所以递推既不会遗漏，也不会重复。全部传输损失为 $2 \sum_{d=1}^r df_d - nm$ ，最后的 nm 对应每个结点损失公式中的 -1 。

正解 1 100 Pts

- ▶ 每一对坐标恰好归入唯一的 f_d ，所以递推既不会遗漏，也不会重复。全部传输损失为 $2 \sum_{d=1}^r d f_d - nm$ ，最后的 nm 对应每个结点损失公式中的 -1 。
- ▶ 内层枚举总次数为 $\sum_{d=1}^r \lfloor r/d \rfloor$ ，时间复杂度为 $\mathcal{O}(r \log r)$ ，空间复杂度为 $\mathcal{O}(r)$ 。乘法、 f_d 与答案都需要使用 long long。

正解 1 100 Pts

```
1  int n,m,r;
2  long long Ans=0,f[MAXN];
3  void Solve() {
4      scanf("%d%d",&n,&m);
5      Ans=0;
6      r=min(n,m);
7      for (int i=r;i>=1;i--) {
8          f[i]=1ll*(n/i)*(m/i);
9          for (int j=2;i*j<=r;j++) {
10             f[i]-=(f[i*j]);
11         }
12         Ans+=1ll*2*i*f[i];
13     }
14     Ans-=1ll*n*m;
15     printf("%lld\n",Ans);
16 }
```

正解 2 100 Pts

- 仍然令 $r = \min(n, m)$ 。定义 $F(d)$ 为满足 $d \mid x$ 且 $d \mid y$ 的有序点对数量，定义 $f(d)$ 为满足 $\gcd(x, y) = d$ 的有序点对数量。于是

$$F(d) = \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor = \sum_{d \mid k} f(k).$$

正解 2 100 Pts

- ▶ 仍然令 $r = \min(n, m)$ 。定义 $F(d)$ 为满足 $d \mid x$ 且 $d \mid y$ 的有序点对数量，定义 $f(d)$ 为满足 $\gcd(x, y) = d$ 的有序点对数量。于是

$$F(d) = \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor = \sum_{d \mid k} f(k).$$

- ▶ 对倍数关系作 Möbius 反演，可以得到

$$f(d) = \sum_{t=1}^{\lfloor r/d \rfloor} \mu(t) F(dt).$$

这一步直接把“最大公约数恰为 d ”写成了容易计算的倍数计数。

正解 2 100 Pts

- 记全部最大公约数之和为 S 。交换枚举顺序，并令 $k = dt$ ，有

$$S = \sum_{d=1}^r df(d) = \sum_{k=1}^r F(k) \sum_{t|k} \mu(t) \frac{k}{t}.$$

正解 2 100 Pts

- 记全部最大公约数之和为 S 。交换枚举顺序，并令 $k = dt$ ，有

$$S = \sum_{d=1}^r df(d) = \sum_{k=1}^r F(k) \sum_{t|k} \mu(t) \frac{k}{t}.$$

- 根据 $\varphi(k) = \sum_{t|k} \mu(t) \frac{k}{t}$ ，最终得到

$$S = \sum_{k=1}^r \varphi(k) \left\lfloor \frac{n}{k} \right\rfloor \left\lfloor \frac{m}{k} \right\rfloor.$$

线性筛预处理 Euler 函数后枚举 k ，原问题的答案为 $2S - nm$ 。时间复杂度与空间复杂度均为 $\mathcal{O}(r)$ ，计算时需要使用 long long。

Contents

T1 五轮启幽扉 (lock)

T2 双径会深庭 (cross)

T3 星脉聚流光 (energy)

T4 重彩书遗年 (paint)

吐槽时间

数据点 1 ~ 6 24 Pts

- 首先考虑一个合法序列从左向右会怎样变化。第一个位置必须为 1，最后一个位置必须为 m 。若前一位不是 m ，当前值只能接着增加到前一位加 1，或者从 1 开始一次新的覆写。若前一位是 m ，覆盖到前一位的操作已经结束，可以调整操作顺序，使当前位置接在任意合法前缀后面。

数据点 1 ~ 6 24 Pts

- ▶ 首先考虑一个合法序列从左向右会怎样变化。第一个位置必须为 1，最后一个位置必须为 m 。若前一位不是 m ，当前值只能接着增加到前一位加 1，或者从 1 开始一次新的覆写。若前一位是 m ，覆盖到前一位的操作已经结束，可以调整操作顺序，使当前位置接在任意合法前缀后面。
- ▶ 枚举保留不修改的位置集合。对一个集合定义布尔状态 $g_{i,j}$ ，表示处理到第 i 位，当前位置最终为 j ，并且集合中不超过 i 的位置均保持原值时是否可行。枚举全部 2^n 个集合，取可行集合大小的最大值，答案为 n 减去这个最大值。

数据点 1 ~ 6 24 Pts

- 令 $w_{i,j}$ 表示第 i 位不要求保留，或者 $a_i = j$ 。初值为 $g_{1,1} = w_{1,1}$ ，其余状态为假。完整转移为

$$\begin{cases} g_{i,1} = \left(\bigvee_{k=1}^m g_{i-1,k} \right) \wedge w_{i,1}, \\ g_{i,j} = (g_{i-1,j-1} \vee g_{i-1,m}) \wedge w_{i,j}, \quad 2 \leq j \leq m. \end{cases}$$

最终要求 $g_{n,m}$ 为真。

数据点 1 ~ 6 24 Pts

- 令 $w_{i,j}$ 表示第 i 位不要求保留，或者 $a_i = j$ 。初值为 $g_{1,1} = w_{1,1}$ ，其余状态为假。完整转移为

$$\begin{cases} g_{i,1} = \left(\bigvee_{k=1}^m g_{i-1,k} \right) \wedge w_{i,1}, \\ g_{i,j} = (g_{i-1,j-1} \vee g_{i-1,m}) \wedge w_{i,j}, \quad 2 \leq j \leq m. \end{cases}$$

最终要求 $g_{n,m}$ 为真。

- 状态逐位枚举了上一页讨论中的全部合法衔接，因此充要性成立。时间复杂度为 $\mathcal{O}(T2^n nm)$ ，空间复杂度为 $\mathcal{O}(m)$ 。瓶颈是重复枚举保留集合，而每个位置是否保留可以直接计入状态值。

数据点 7 ~ 12 48 Pts

- 根据上一档部分分的启发，把布尔状态改成最大保留位置数量。定义 $f_{i,j}$ 表示处理到第 i 位，当前位置最终为 j 时，最多有多少个位置不需要修改。 $m \leq 2$ 时只有两个状态，直接按照上一页的两种衔接更新。

数据点 7 ~ 12 48 Pts

- ▶ 根据上一档部分分的启发，把布尔状态改成最大保留位置数量。定义 $f_{i,j}$ 表示处理到第 i 位，当前位置最终为 j 时，最多有多少个位置不需要修改。 $m \leq 2$ 时只有两个状态，直接按照上一页的两种衔接更新。
- ▶ 不可能的状态设为负无穷，初值为 $f_{1,1} = [a_1 = 1]$ ，其余状态为负无穷，答案为 $n - f_{n,m}$ 。每个位置只进行常数转移，时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(1)$ 。

数据点 13 ~ 19 76 Pts

- 去掉 $m \leq 2$ 的限制后仍然沿用同一状态。用 Iverson 括号表示当前位置是否可以保留，完整转移为

$$\begin{cases} f_{i,1} = \max_{1 \leq k \leq m} f_{i-1,k} + [a_i = 1], \\ f_{i,j} = \max\{f_{i-1,j-1}, f_{i-1,m}\} + [a_i = j], \quad 2 \leq j \leq m. \end{cases}$$

数据点 13 ~ 19 76 Pts

- 去掉 $m \leq 2$ 的限制后仍然沿用同一状态。用 Iverson 括号表示当前位置是否可以保留，完整转移为

$$\begin{cases} f_{i,1} = \max_{1 \leq k \leq m} f_{i-1,k} + [a_i = 1], \\ f_{i,j} = \max\{f_{i-1,j-1}, f_{i-1,m}\} + [a_i = j], \quad 2 \leq j \leq m. \end{cases}$$

- 初始化、负无穷状态与答案位置均和上一档相同。滚动数组后空间复杂度为 $\mathcal{O}(m)$ ，时间复杂度为 $\mathcal{O}(nm)$ 。正解只需要优化右移、全局最大值、区间取最大值与单点加这四类重复操作。

正解 100 Pts

- ▶ 上述局部转移等价于把合法序列划分成若干连续上升段。相邻两段之间，要么前一段以 m 结尾，要么后一段以 1 开头。必要性来自相邻两次覆写的先后关系。若左侧覆写较晚，前一段会延伸到它写下的 m 。若右侧覆写较晚，后一段会从它写下的 1 开始。

正解 100 Pts

- ▶ 上述局部转移等价于把合法序列划分成若干连续上升段。相邻两段之间，要么前一段以 m 结尾，要么后一段以 1 开头。必要性来自相邻两次覆写的先后关系。若左侧覆写较晚，前一段会延伸到它写下的 m 。若右侧覆写较晚，后一段会从它写下的 1 开始。
- ▶ 反过来，从左到右按照转移构造。段内使用 $j-1 \rightarrow j$ ，新段从 1 开始，以 m 结束的段后允许接入任意合法状态。每一种满足分段条件的序列都能由这些转移得到，所以条件也是充分的，DP 没有加入题目中无法得到的序列。

正解 100 Pts

- 在处理第 i 位之前, 令线段树坐标 $n1 + j - 1$ 保存 $f_{i-1,j}$ 。先查询旧的 f_m 记为 Last, 再令 $n1 \leftarrow n1 - 1$ 。这样旧的 f_{j-1} 自动落到新的状态 f_j 所在坐标, 完成所有状态整体右移。

正解 100 Pts

- ▶ 在处理第 i 位之前，令线段树坐标 $n1 + j - 1$ 保存 $f_{i-1,j}$ 。先查询旧的 f_m 记为 Last，再令 $n1 \leftarrow n1 - 1$ 。这样旧的 f_{j-1} 自动落到新的状态 f_j 所在坐标，完成所有状态整体右移。
- ▶ 新的 f_1 等于旧状态最大值 Best，在坐标 $n1$ 单点写入。对 $j = 2, \dots, m$ 统一执行 $f_j \leftarrow \max(f_j, \text{Last})$ ，对应区间取最大值。最后在坐标 $n1 + a_i - 1$ 单点加 1，并用返回值更新 Best。

正解 100 Pts

- ▶ 线段树维护区间最大值，支持区间取最大值与单点加。懒标记 Tag 表示整段至少取到的下界，两个标记的复合就是取最大值。初始时全部位置为负无穷，只在表示 $f_{1,1}$ 的位置写入 $[a_1 = 1]$ ，最终查询表示 $f_{n,m}$ 的位置。

正解 100 Pts

- ▶ 线段树维护区间最大值，支持区间取最大值与单点加。懒标记 Tag 表示整段至少取到的下界，两个标记的复合就是取最大值。初始时全部位置为负无穷，只在表示 $f_{1,1}$ 的位置写入 $[a_1 = 1]$ ，最终查询表示 $f_{n,m}$ 的位置。
- ▶ 每个位置进行常数次数段树操作，时间复杂度为 $\mathcal{O}(n \log(n+m))$ ，空间复杂度为 $\mathcal{O}(n+m)$ 。坐标范围开到 $n+m+10$ ，保证逻辑起点不断左移后全部 m 个状态仍在线段树中。

正解 100 Pts

```
1 inline void Modify(int Now,int l,int r,int Val) {
2     if (l>r) return;
3     if (Node[Now].l>r | Node[Now].r<l) return;
4     else if (Node[Now].l>=l && Node[Now].r<=r) Node[Now].Tag=max(Node[Now]
5         ].Tag,Val),Node[Now].Max=max(Node[Now].Max,Val);
6     else {
7         PushDown(Now);
8         Modify(LSon,l,r,Val);Modify(RSon,l,r,Val);Update(Now);
9         return;
10    }
```

正解 100 Pts

```
1 inline int Add(int Now,int Pos) {
2     int Ret;
3     if (Node[Now].l==Node[Now].r) {Node[Now].Max++;return Node[Now].Max
4         ;}
5     else {
6         PushDown(Now);
7         if (Node[Lson].r>=Pos) Ret=Add(Lson,Pos);
8         else Ret=Add(Rson,Pos);
9         Update(Now);
10    }
11    return Ret;
12 }
```

正解 100 Pts

```
1 inline int Query(int Now,int l,int r) {
2     if (l>r) return -0x3f3f3f3f;
3     if (Node[Now].l>r||Node[Now].r<l) return -0x3f3f3f3f;
4     else if (Node[Now].l>=l&&Node[Now].r<=r) return Node[Now].Max;
5     else {
6         PushDown(Now);Update(Now);
7         return max(Query(LSon,l,r),Query(RSon,l,r));
8     }
9 }
```


正解 100 Pts

```
1 void Solve() {
2     scanf("%d%d",&n,&m);
3     for (int i=1;i<=n;i++) {
4         scanf("%d",&Line[i]);
5     }
6     int nl=n;cnt=0;
7     Build(1,n+m+10);
8     Modify(1,nl,nl,(Line[1]==1));
9     int Best=(Line[1]==1);
10    for (int i=2;i<=n;i++) {
11        int Last=Query(1,nl+m-1,nl+m-1);
12        nl--;
13        Modify(1,nl,nl,Best);
14        Modify(1,nl+1,nl+m-1,Last);
15        int NewVal=Add(1,nl-1+Line[i]);
16        Best=max(Best,NewVal);
17    }
18    printf("%d\n",n-Query(1,nl+m-1,nl+m-1));
19    return ;
```

Contents

T1 五轮启幽扉 (lock)

T2 双径会深庭 (cross)

T3 星脉聚流光 (energy)

T4 重彩书遗年 (paint)

吐槽时间